

The background of the slide is a light beige color with a faint grid pattern. Overlaid on this grid are several architectural drawings in a light brown or tan color. These drawings include floor plans, cross-sections, and elevations of various structures, possibly bridges or industrial buildings. Some drawings have dimension lines with numbers, though they are small and difficult to read. The drawings are scattered across the slide, with some being more prominent than others.

The 45-Minute System Design Playbook

A repeatable, time-boxed framework
for scaling from zero to billions.

Featuring the YouTube Architecture Case Study

The Unstructured Trap



Memorizing specific architectures.

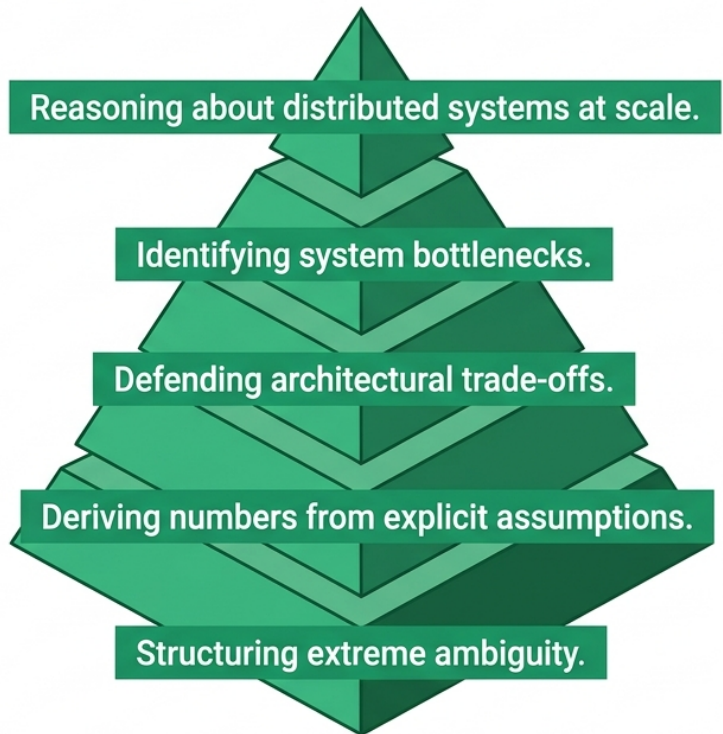
Jumping straight to databases.

Over-engineering for day-one traffic.

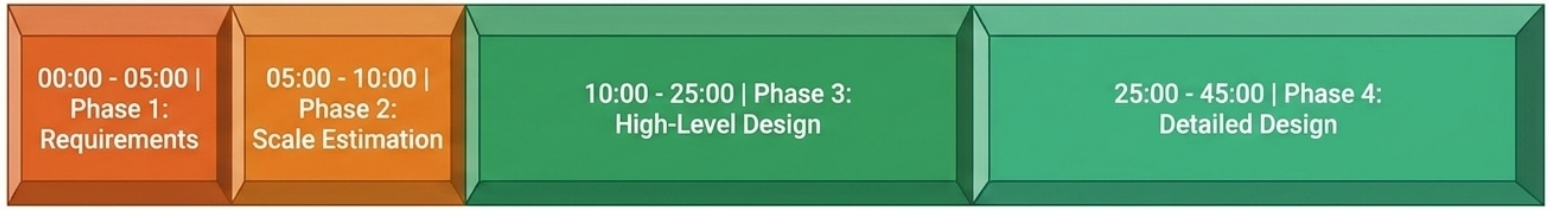
Wasting time on low-value components.

Running out of time before discussing trade-offs.

The Evaluated Signals



The 45-Minute Algorithm



Do not treat the interview as a conversation. Treat it as a time-boxed algorithm. The framework forces prioritization, preventing premature optimization and ensuring critical failure scenarios are addressed.

Phase 1: Scope the Problem (Strict 5-Minute Limit)

Functional (What it does)

Limit to 3-4 core user journeys.

Example: Users create content, interact with content, search content.

The Golden Rule

Ask **ONLY** the questions that materially alter the architecture. Do not ask for user demographics if it doesn't change data routing. High read/write ratios, latency expectations, and global distribution are the only variables that dictate CDN, replication, and data residency choices.

Non-Functional (How it performs)

Define Latency, Availability, Consistency, Durability.

Map geographic distribution (Who/Where).

YouTube Application: Extracting Core Constraints

Functional Constraints

1. Upload videos
2. Stream videos
3. Search by title
4. View thumbnails

Non-Functional Constraints

High Availability (99%+)

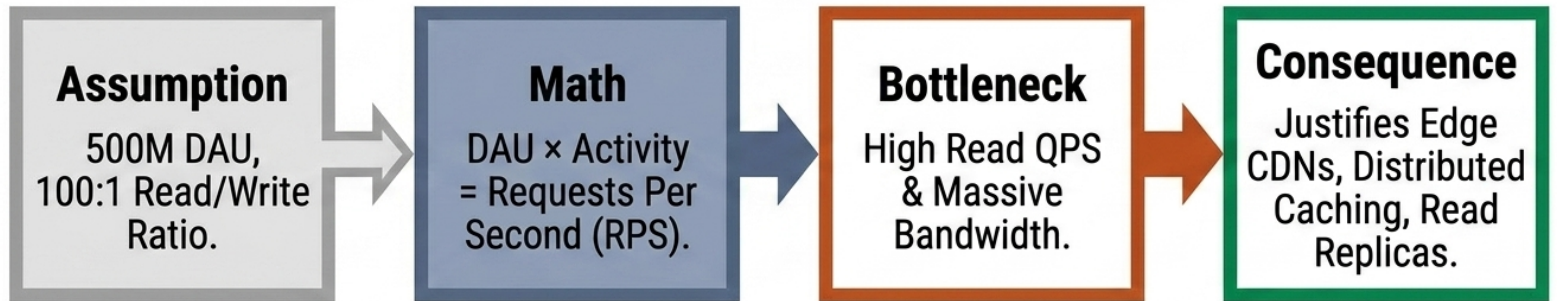
Minimal Latency (No buffering)

Eventual Consistency (Video feeds don't require instant strong consistency)



At 2.5B users, scalability outranks strong consistency.

Phase 2: The Consequence Pipeline



Never invent random figures. Derive numbers explicitly. If the math doesn't change the design, don't do the math.

YouTube Case Study: Storage Estimation

Total Uploads = 500 hours/minute

Storage Cost = 6 MB per minute of compressed video (30MB per 5-min video)

Minutes = $500 \times 60 = 30,000$ minutes of video

Total Storage = $30,000 \times 6 \text{ MB} = 180 \text{ GB/minute}$

Result: 180 GB of new data per minute.

Architectural Consequence: Relational databases cannot store this payload. Forces the use of distributed Blob Storage (like GFS/S3) for video files, keeping only lightweight metadata in the database.

YouTube Case Study: Bandwidth Estimation

The **Asymmetry Assumption**: Upload : View Ratio = 1 : 300 (For every 1 uploaded video, 300 are viewed).

Upload

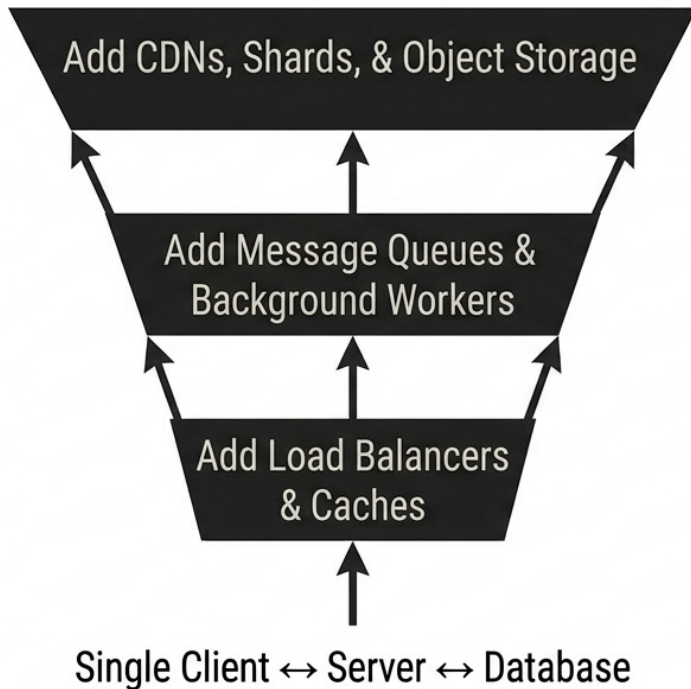
500 hrs/min uploaded
120 MB/min
transmission size
Result: 480 Gbps
required just for
uploads.

Stream

If uploads require 480 Gbps,
streaming requires ~300x that capacity.

Architectural Consequence: This extreme read-heavy bandwidth asymmetry makes centralized data centers physically impossible. Content **MUST** be pushed to edge networks (CDNs, ISPs).

Phase 3: The Incremental Architecture Rule



Concept

Start with a single-user baseline. Only introduce distributed-system complexity when the scale estimation explicitly demands it.

Checklist for every component:

1. What specific problem does this solve?
2. Why is it needed at our calculated scale?
3. What happens if this node fails?

YouTube Case Study: Core APIs

Upload Path

```
1 | POST /video/upload
2 | Payload: {
3 |     user_id,
4 |     video_file,
5 |     title,
6 |     default_language
7 | }
```

Note: Video files are split into smaller packets and uploaded sequentially. If failure occurs, data is retained temporarily to resume.

Stream Path

```
1 | GET /video/stream
2 | Params: {
3 |     video_id,
4 |     screen_resolution,
5 |     user_bitrate,
6 |     device_chipset
7 | }
```

Note: user_bitrate and device_chipset are critical for the server to optimize quality and select the correct video chunks based on client transmission capacity.

Phase 3: Decoupled Storage Schema



User & Metadata. Structured user data and video metadata (Title, Description, Tags) to support efficient search and relationship mapping. Handles limited concurrency.

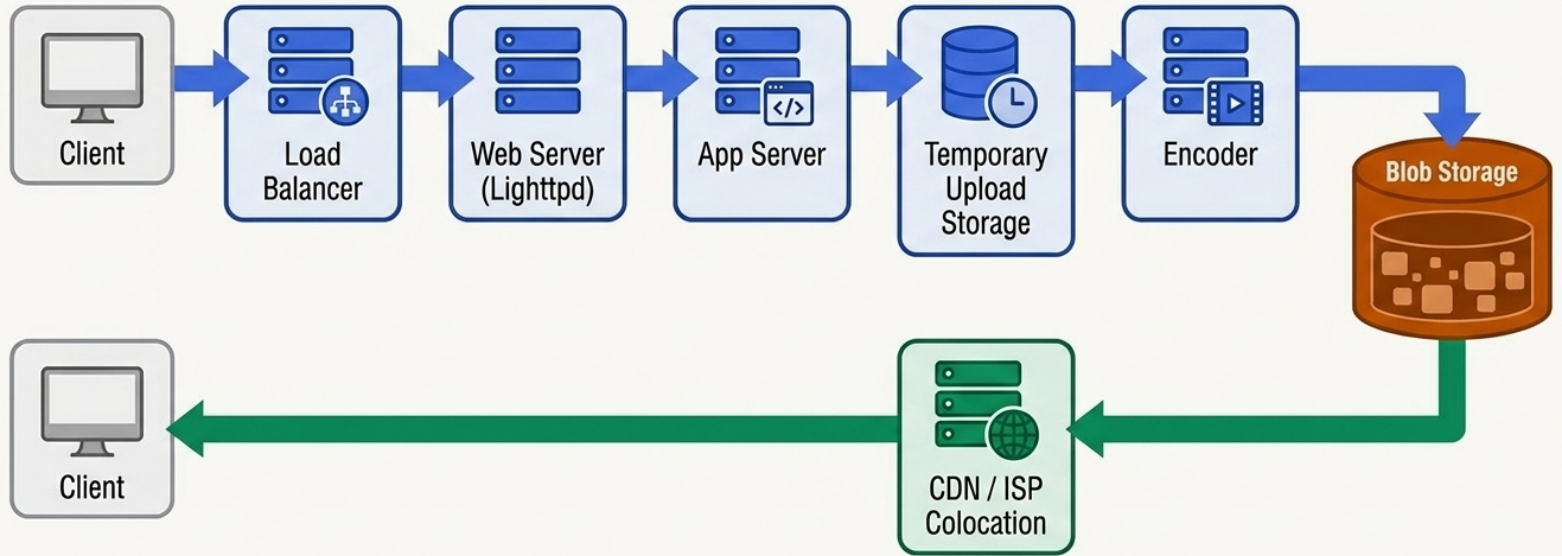


Thumbnails. High throughput key-value store based on Google File System. Ideal for storing metadata and references for billions of thumbnails with low-latency access.



Video Content. Scalable object storage for raw and encoded video files.

YouTube End-to-End High-Level Architecture



Design Choice: Web servers handle static content rapidly, decoupling client requests from App servers that execute heavy business logic. Encoders run asynchronously in the background to generate multiple resolutions without blocking the upload response.

Phase 4: Anticipate and Defend Trade-offs

1. Consistency vs. Availability

Apply CAP Theorem logic. Eventual consistency is fine for video feeds, but strong consistency is strictly needed for billing or user authentication.

2. Bottleneck Identification

Where does the system break at 10x scale? In write-heavy scenarios, it is usually the database write-path.

3. SPOF & Failure Scenarios

What happens if a data center goes dark? Mitigate with heartbeat protocols, consistent hashing, and multi-region replication.

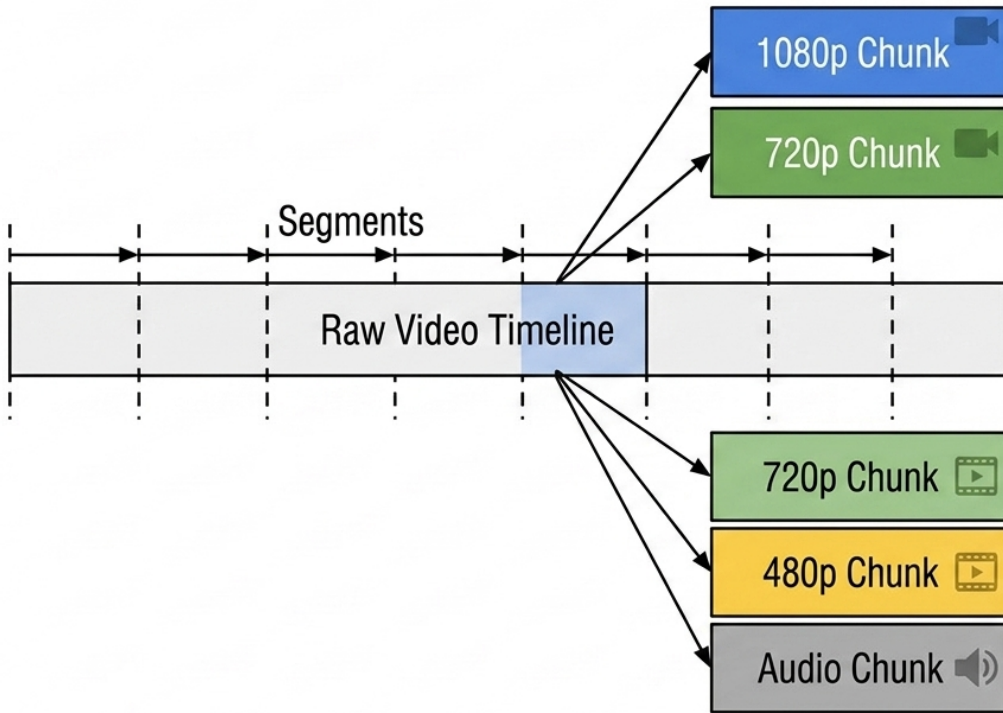
4. Caching Strategies

Where are we caching? Deploy Distributed Memcached with Least Recently Used (LRU) eviction for long-tail access patterns.

Critical Trade-off Matrices

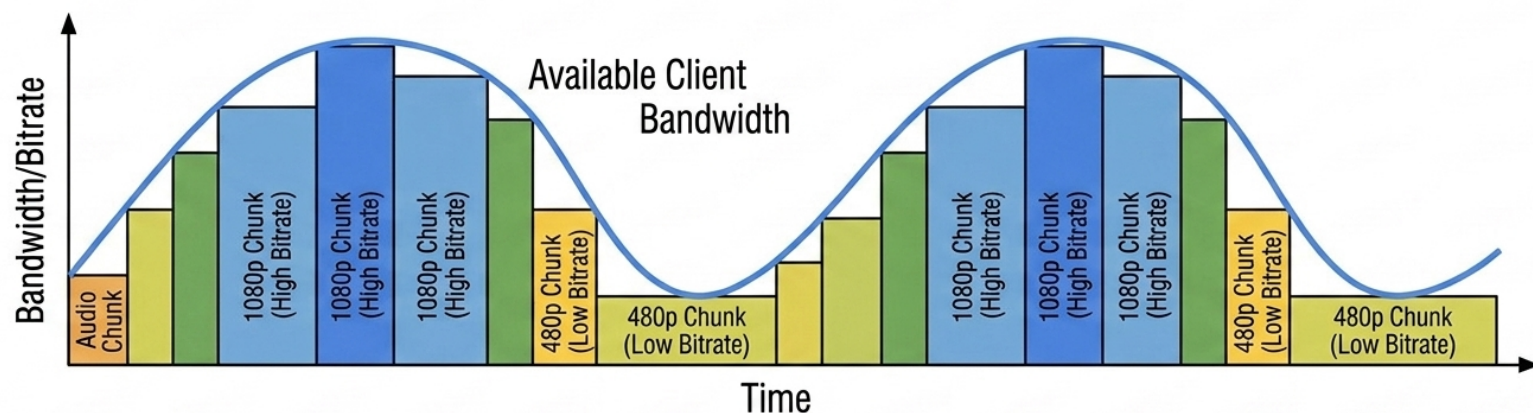
Content Delivery Network (CDN) Strategy	
Public CDN	Low CAPEX. Ideal for sudden viral traffic in remote regions. Becomes prohibitively expensive at sustained global scale.
Private CDN / Colocation	High initial setup. Placed directly in ISP Point of Presence (PoPs). Required for YouTube's massive sustained baseline traffic.
Caching & Serving Optimizations	
Flash Servers	Hold popular/viral content. Optimized exclusively for low-latency delivery and speed.
Storage Servers	Hold the 'long tail' of unpopular videos. Optimized for cheap, massive storage capacity over speed.

Deep Dive: Per-Shot Encoding & Chunking



- **Concept:** We do not encode an entire video as one file. We divide the video into short time frames (**segments**).
- **The Optimization: Dynamic scenes** get less compression; static scenes are **heavily compressed** to save storage.
- **The Result:** Each segment is encoded into **multiple chunks** of varying bitrates. This **per-shot granularity** **dramatically reduces storage overhead** and allows a **larger catalog** to fit into ISP edge caches.

Deep Dive: Adaptive Streaming Delivery



Mechanism

The client device continuously measures end-to-end available bandwidth and its own buffer space.

Action

When bandwidth drops, the client **requests a smaller, lower-bitrate chunk**. When bandwidth improves, it requests a high-resolution chunk.

Takeaway

Intelligence is pushed to the **client**. The server simply provides **multiple chunk options**; the client dynamically adapts to prevent buffering.

Optimization at Petabyte Scale: Deduplication

9.5 Petabytes

Storage saved annually by identifying duplicate uploads.

(Math: 50 duplicate hrs/min × 60 mins × 6 MB = **18 GB/min** wasted space).

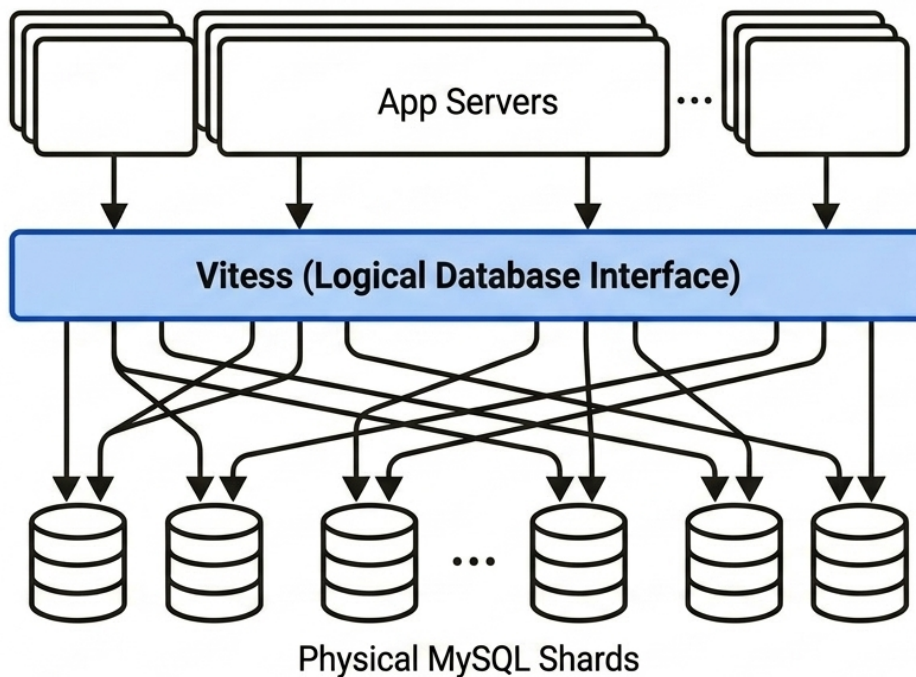
Duplicate Detection Pipeline

Locality-Sensitive Hashing (LSH): Fast, cryptographic-style hashing used to find exact or near-exact simple matches.

Block Matching Algorithms (BMA): Compares localized blocks of pixels to detect altered or cropped duplicates.

Machine Learning Pipelines: Deployed for complex copyright infringement detection beyond simple bit-matching.

Deep Dive: Scaling MySQL with Vitess



The Problem:

As metadata grows, a single MySQL instance bottlenecks. Manual sharding breaks ACID properties and leaks complex routing logic into the application code.

The Solution:

Developed by YouTube, Vitess presents a single logical interface to the app. It automatically manages sharding and maintains transactional (ACID) guarantees.

Bonus Insight:

Google utilizes custom ESF (Extensible Service Proxy) web servers to squeeze out performance that off-the-shelf software cannot provide.

The Master Blueprint

Phase	Whiteboard Artifact	Verbal Narrative
0-5m (Requirements)	Bulleted list of 3-4 features + NFRs	I am limiting scope to X and prioritizing availability over consistency.
5-10m (Scale Estimation)	Back-of-envelope math + variables	Assuming 1:300 write/read ratio, we are bottlenecked by CDN egress.
10-25m (High-Level Design)	Monospace APIs + Storage schema + End-to-End Diagram	I'll start with a single user flow, then decouple read/write paths.
25-45m (Detailed Design)	Zoomed-in diagrams of bottlenecks (e.g., Sharding/Vitess, Chunking)	If this node fails, the heartbeat protocol triggers consistent hash rebalancing.

Print this blueprint. Treat it as your mental algorithm for every system design interview.